Day 2 Recap

What We Learned Today

KavinSchool

Day 2: What We Covered

Session 1: Prompt Engineering Fundamentals

- The 4 building blocks: Instruction, Context, Input Data, Output Format
- Common anti-patterns and how to avoid them
- System message vs user message responsibilities

Session 2: Prompting Techniques

- Zero-shot, few-shot, Chain-of-Thought, role prompting
- Structured output strategies (prompt-based, JSON mode)
- Combining techniques for effective prompts

Day 2: What We Covered

Session 3: API Parameters

- temperature, top_p, frequency/presence penalty
- max_tokens, stop sequences, seed
- Streaming responses for real-time UX

Session 4: Iteration & Refinement

- The write → test → analyze → refine loop
- Evaluation: manual rubrics, golden datasets, LLM-as-judge
- Prompt versioning and configuration

Day 3 Preview: Building Custom Chatbots & RAG

Tomorrow we build **real applications**:

- **Conversation context:** managing multi-turn chats at scale
- **RAG (Retrieval-Augmented Generation):** ground LLM responses in your data
- **Vector databases:** FAISS, Chroma for similarity search
- **End-to-end RAG chatbot:** ingest docs, embed, retrieve, generate
- **API wrappers:** Flask/FastAPI endpoints for your chatbot

> **Homework (optional):** Take your Lab 2.5 Refactor Assistant and add 5 more test cases. Can you get it to handle edge cases like global variables, nested functions, and class methods?

Quick Knowledge Check

1. What are the **4 building blocks** of an effective prompt?
2. When would you use **few-shot** over **zero-shot** prompting?
3. What temperature would you use for **JSON extraction**?
4. What's the difference between `frequency_penalty` and `presence_penalty`?
5. Why is **prompt versioning** important?
6. Name **two ways** to evaluate prompt output quality.

Thank You!

Day 2 Complete

See you tomorrow for **Custom Chatbots & RAG!**

KavinSchool